Parking Spot Detection Using YOLO11

A Computer Vision Approach with Synthetic Data Augmentation

Esad Kaya
Student Number: 300283659
ekaya@uOttawa.ca

March 3, 2026

SEG 4180: Applied Machine Learning
Professor Daniel Shapiro
University of Ottawa

Abstract

This report documents the development of a parking spot detection system using the YOLO11 object detection architecture. The system is designed to identify vehicles and parking enforcement vehicles in street-level imagery obtained from Google Street View. We describe the data collection pipeline, preprocessing steps, and the machine learning workflow employed. The project leverages synthetic data generation via Google's Gemini API to augment the training dataset, addressing the challenge of limited labeled data for parking enforcement vehicles.

Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Objectives . . . 3
1.3 Scope . . . 3

2 Methodology **3**
2.1 System Architecture . . . 3
2.2 Tools and Technologies . . . 4

3 Data Collection **4**
3.1 Overview . . . 4
3.2 Location Sampling . . . 4
3.2.1 Points Manifest . . . 4
3.2.2 Geographic Coverage . . . 4
3.3 Image Retrieval . . . 5
3.3.1 API Integration . . . 5
3.3.2 Directory Structure . . . 5
3.3.3 Resume Capability . . . 5
3.3.4 Execution Results . . . 5

3.4 Data Quality Assurance . . . 6
3.4.1 Duplicate Detection . . . 6
3.4.2 Resolution . . . 6
3.4.3 Final Dataset Statistics . . . 6

4 Image Preprocessing **6**
4.1 Motivation . . . 6
4.2 Implementation . . . 7
4.2.1 Algorithm . . . 7
4.2.2 Idempotency . . . 7
4.2.3 File Preservation . . . 7
4.3 Results . . . 8

5 Empty Frame Detection **8**
5.1 Overview . . . 8
5.2 Method . . . 8
5.3 Results . . . 8

6 Synthetic Data Generation **9**
6.1 Overview . . . 9
6.2 Method . . . 9
6.3 Console Output . . . 9
6.4 Output . . . 9
6.5 Next Steps . . . 9

7 Label Generation **10**
7.1 Automated Annotation . . . 10
7.2 Manual Review . . . 10

8 Dataset Preparation **10**
8.1 Train/Validation/Test Split . . . 10

9 Model Training **10**
9.1 Configuration . . . 10
9.2 Training Metrics . . . 10

10 Evaluation **11**
10.1 Metrics . . . 11
10.2 Per-Class Performance . . . 11

11 Discussion **11**
11.1 Challenges . . . 11
11.2 Lessons Learned . . . 11
11.3 Future Work . . . 11

12 Conclusion **12**

1 Introduction

1.1 Motivation

Parking enforcement is a critical urban management task that traditionally requires significant human labor. Automating the detection of parking violations—particularly identifying parking enforcement vehicles in restricted zones—presents an opportunity to improve efficiency and consistency in municipal operations.

Computer vision techniques, particularly object detection models, have advanced significantly in recent years. The YOLO (You Only Look Once) family of models offers real-time detection capabilities suitable for deployment in practical applications.

1.2 Objectives

The primary objectives of this project are:

1. Develop a data collection pipeline for acquiring street-level imagery
2. Implement preprocessing steps to prepare images for machine learning
3. Train a YOLO11 model to detect four classes: regular vehicles, parking enforcement vehicles, and two types of police cruisers
4. Evaluate model performance and document findings

1.3 Scope

This project focuses on the Ottawa, Ontario area for data collection. The model is trained to detect:

- **Class 0: vehicle** — Regular vehicles (cars, trucks, buses, motorcycles)
- **Class 1: enforcement_vehicle** — Parking enforcement vehicles
- **Class 2: police_old** — Ottawa Police cruisers with old livery
- **Class 3: police_new** — Ottawa Police cruisers with new livery

2 Methodology

2.1 System Architecture

The project follows a modular pipeline architecture:

1. **Data Collection**: Automated fetching of Google Street View images
2. **Preprocessing**: Image cropping and quality filtering
3. **Empty Frame Detection**: Identification of frames without vehicles
4. **Synthetic Data Generation**: Insertion of vehicles into empty frames

5. **Label Generation**: Automated bounding box annotation
6. **Dataset Splitting**: Train/validation/test partitioning
7. **Model Training**: YOLO11 fine-tuning
8. **Evaluation**: Performance metrics and analysis

2.2 Tools and Technologies

Component	Technology
Object Detection	YOLO11s (Ultralytics)
Deep Learning Framework	PyTorch
Image Processing	Pillow (PIL)
Data Augmentation	Google Gemini API
Geographic Data	Google Street View Static API
Programming Language	Python 3.12
Version Control	Git

Table 1: Technology stack overview

3 Data Collection

3.1 Overview

The data collection phase involved systematically sampling locations within Ottawa and retrieving Google Street View imagery from each location at multiple heading angles.

3.2 Location Sampling

3.2.1 Points Manifest

A points manifest (`manifests/points.csv`) was created containing 9,600 entries representing 1,200 unique panorama locations. Each panorama was captured at 8 heading angles (0°, 45°, 90°, 135°, 180°, 225°, 270°, 315°) to provide comprehensive street-level coverage.

The manifest schema includes:

- `street`: Unique identifier for the location
- `label`: Human-readable label combining panorama ID and heading
- `location`: Latitude and longitude coordinates
- `heading`: Compass direction (0–360°)
- `pitch`: Camera tilt angle
- `fov`: Field of view (90°)
- `pano_id`: Google Street View panorama identifier

3.2.2 Geographic Coverage

The sampled locations cover various urban environments in Ottawa, including:

- Downtown commercial areas
- Residential neighborhoods
- Parking lots and street parking zones

3.3 Image Retrieval

3.3.1 API Integration

Images were retrieved using the Google Street View Static API. A custom Python script (`scripts/02_fetch_streetview.py`) was developed to:

1. Read the points manifest
2. Construct API requests with appropriate parameters
3. Download and store images in a structured directory hierarchy
4. Record metadata in an images manifest (`manifests/images.csv`)

3.3.2 Directory Structure

Images were organized by panorama and heading:

```
data/images_original/
+-- auto_{pano_id}_h0/
|   +-- {hash}.jpg
+-- auto_{pano_id}_h45/
|   +-- {hash}.jpg
+-- ...
```

Each image filename is a random hash, while the parent directory encodes the panorama ID and heading angle.

3.3.3 Resume Capability

The fetch script implements a resume mechanism using the `-resume` flag. It checks the existing images manifest and only downloads images not already present, enabling recovery from interrupted sessions.

3.3.4 Execution Results

The data collection completed successfully with the following statistics:

Metric	Value
Total images downloaded	9,600
Unique panoramas	1,200
Headings per panorama	8
Total duration	33 minutes 50 seconds
Average download rate	1.78 images/second
Manifest entries	9,600

Table 2: Data collection execution statistics

The manifest was saved to `manifests/images.csv` with all 9,600 entries successfully recorded.

3.4 Data Quality Assurance

3.4.1 Duplicate Detection

A critical issue emerged during data collection: duplicate images were present in the dataset. Two types of duplicates were identified:

1. **Panorama duplicates**: The same panorama ID appearing multiple times in the manifest
2. **File duplicates**: Multiple files in the same directory (same panorama and heading) with identical content

Panorama-Level Verification Using PowerShell and Python scripts, we verified that the points manifest contained exactly 1,200 unique panorama IDs, each with 8 heading entries. No duplicate panoramas were found in the manifest.

File-Level Verification The `jdupes` tool was used to identify byte-identical files. The analysis revealed:

- One panorama (`CcKfN3NH1TTKafS0fsXcjA`) had 2 files in 7 of its 8 heading directories
- These duplicates resulted from overlapping data between a previous ZIP archive and the current fetch operation

3.4.2 Resolution

Duplicates were removed using:

```
jdupes -r -d -N data/images_original
```

This command recursively scans, identifies duplicates, preserves the first file in each duplicate set, and deletes the rest without prompting.

3.4.3 Final Dataset Statistics

Metric	Count
Unique panoramas	1,200
Heading angles per panorama	8
Total images	9,600
Images manifest entries	9,600

Table 3: Final dataset statistics after cleanup

4 Image Preprocessing

4.1 Motivation

Google Street View images often contain visual elements at the bottom of the frame that are not relevant to parking detection, including:

- Copyright watermarks
- Timestamp overlays
- Navigation UI elements

These elements can interfere with model training and should be removed.

4.2 Implementation

A cropping script (`scripts/02b_crop_bottom.py`) was developed to remove the bottom 30 pixels from each image.

4.2.1 Algorithm

Algorithm 1 Bottom Crop Algorithm

```
for each row in images manifest do
    if status is not "ok" then
        Skip row
    end if
    if filename contains "_bcN" suffix then
        Skip (already cropped)
    end if
    if cropped file exists on disk then
        Skip (already cropped)
    end if
    Load source image
    Crop to (0, 0, width, height − crop_px)
    Save as image_bc30.jpg
    Update manifest with new file path
end for
```

4.2.2 Idempotency

The script is designed to be idempotent—running it multiple times produces the same result. It detects already-cropped images through:

1. Filename suffix matching (`_bc30`)
2. Target file existence checking

4.2.3 File Preservation

Original images are preserved on disk. The cropped version is saved as a sibling file with a `_bc30` suffix:

```
Original: data/images_original/.../abc123.jpg
Cropped:  data/images_original/.../abc123_bc30.jpg
```

The manifest `file_path` column is updated to reference the cropped version, while `source_file_path` retains the original path.

4.3 Results

The cropping operation was executed with the following results:

Metric	Count
Total rows in manifest	9,600
Eligible for cropping	9,600
Newly cropped	3,764
Skipped (already cropped)	5,836
Skipped (missing file)	0
Skipped (bad status)	0
Skipped (synthetic)	0
Failed	0

Table 4: Cropping operation statistics

The idempotent design proved effective: 5,836 images had been cropped in a previous run and were correctly identified and skipped. The remaining 3,764 images (from the most recent fetch) were cropped successfully. Each image now has:

- Original file (preserved)
- Cropped file (used for training)

5 Empty Frame Detection

5.1 Overview

Empty frame detection identifies images that contain no vehicles. These frames are valuable for:

- Synthetic data generation (clean backgrounds for vehicle insertion)
- Negative samples for training

5.2 Method

- Model used: YOLO11s pretrained on COCO
- Confidence threshold: [TO BE DETERMINED]
- Vehicle classes detected: car, truck, bus, motorcycle

5.3 Results

- Total frames analyzed: [TO BE DETERMINED]
- Empty frames identified: [TO BE DETERMINED]
- Output: `lists/empty_candidates.txt`

6 Synthetic Data Generation

6.1 Overview

Synthetic data generation addresses the scarcity of parking enforcement vehicle images by using generative AI to insert vehicles into empty frames.

6.2 Method

The synthesis pipeline used Google's Gemini API with the following configuration:

- API: Google Gemini (gemini-3-pro-image-preview)
- Edit types: Vehicle insertion into empty frames
- Classes generated: enforcement_vehicle, police_old, police_new
- Image resizing: Cover-resize + center-crop to (640, 610)

6.3 Console Output

The synthesis completed successfully after processing all candidates:

```
Synthesizing: 100%|======...======| 1471/1471 [1:11:33<00:00]
2026-03-03 03:48:11 - INFO - Created 2658 synthetic images
2026-03-03 03:48:11 - INFO - Manifest updated: manifests/images.csv
```

6.4 Output

The synthesis produced the following results:

Metric	Value
Input empty frames	1,471
Synthetic images created	2,658
Generation time	1 hour 11 minutes
Output location	data/images_synth/

Table 5: Synthetic data generation statistics

Cost Efficiency: The entire synthesis operation cost only $6.23 in Gemini API credits, demonstrating the cost-effectiveness of generative AI for dataset augmentation compared to traditional data collection methods.

6.5 Next Steps

Following synthesis, the dataset requires manual review:

1. Review synthetic images for quality and realism
2. Delete nonsensical or failed generations
3. Verify bounding box labels match inserted vehicles

7 Label Generation

7.1 Automated Annotation

- Method: [TO BE DETERMINED]
- Output format: YOLO format (class x_center y_center width height)
- Location: `data/labels_autogen/`

7.2 Manual Review

- Tool: Custom web-based labeling UI
- Command: `make labeler`
- URL: `http://localhost:8000`

8 Dataset Preparation

8.1 Train/Validation/Test Split

- Split ratios: [TO BE DETERMINED]
- Grouping strategy: By panorama ID (prevents data leakage)
- Output: `data/splits/`

9 Model Training

9.1 Configuration

- Base model: YOLO11s
- Epochs: [TO BE DETERMINED]
- Batch size: [TO BE DETERMINED]
- Image size: [TO BE DETERMINED]
- Hardware: [TO BE DETERMINED]

9.2 Training Metrics

- Training loss: [TO BE DETERMINED]
- Validation mAP: [TO BE DETERMINED]
- Training time: [TO BE DETERMINED]

10 Evaluation

10.1 Metrics

- Precision: [TO BE DETERMINED]
- Recall: [TO BE DETERMINED]
- mAP@0.5: [TO BE DETERMINED]
- mAP@0.5:0.95: [TO BE DETERMINED]

10.2 Per-Class Performance

Class	Precision	Recall	mAP@0.5
vehicle	[TBD]	[TBD]	[TBD]
enforcement_vehicle	[TBD]	[TBD]	[TBD]
police_old	[TBD]	[TBD]	[TBD]
police_new	[TBD]	[TBD]	[TBD]

Table 6: Per-class detection performance

11 Discussion

11.1 Challenges

- Duplicate data management
- Limited enforcement vehicle samples

BE IDENTIFIED

11.2 Lessons Learned

- Importance of idempotent pipeline design
- Value of comprehensive duplicate checking

BE IDENTIFIED

11.3 Future Work

- Expand geographic coverage
- Improve synthetic data quality
- Deploy as real-time detection system
- Investigate model quantization for edge deployment

12 Conclusion

This project demonstrates the application of modern object detection techniques to the domain of parking enforcement. The modular pipeline architecture enables systematic data collection, preprocessing, and model development. [CONCLUSION TO BE COMPLETED]

References

References

[1] Ultralytics. *YOLO11*. https://docs.ultralytics.com/models/yolo11/ [Accessed: 2026-03-01]

[2] Google. *Street View Static API*. https://developers.google.com/maps/documentation/streetview/ [Accessed: 2026-03-01]

[3] Google. *Gemini API*. https://ai.google.dev/docs [Accessed: 2026-03-01]

[4] PyTorch. *PyTorch Documentation*. https://pytorch.org/docs/ [Accessed: 2026-03-01]

Appendix A: Project Structure

```
parkopticon-yolo11s-bootstrap/
+-- scripts/
|   +-- 01_make_points_template.py
|   +-- 01b_expand_points_by_area.py
|   +-- 02_fetch_streetview.py
|   +-- 02b_crop_bottom.py
|   +-- 03_detect_empty_frames.py
|   +-- 04_synthesize_vehicle_edits.py
|   +-- 05_prelabel_boxes.py
|   +-- 06_split_dataset.py
|   +-- 07_train_yolo.py
|   +-- 08_eval_report.py
+-- labeler/
|   +-- app.py
|   +-- static/
+-- data/
|   +-- images_original/
|   +-- images_synth/
|   +-- labels_autogen/
|   +-- labels_final/
|   +-- splits/
+-- manifests/
|   +-- points.csv
|   +-- images.csv
+-- reports/
+-- dataset.yaml
+-- requirements.txt
+-- Makefile
+-- .env
```

Appendix B: Commands Reference

Operation	Command
Fetch images	`python scripts/02_fetch_streetview.py`
Crop images	`python scripts/02b_crop_bottom.py`
Detect empty	`python scripts/03_detect_empty_frames.py`
Synthesize	`python scripts/04_synthesize_vehicle_edits.py`
Prelabel	`python scripts/05_prelabel_boxes.py`
Launch labeler	`make labeler`
Split dataset	`python scripts/06_split_dataset.py`
Train model	`python scripts/07_train_yolo.py`
Evaluate	`python scripts/08_eval_report.py`

Table 7: Pipeline commands reference